Manipulação de Dados, Validações e Ciclo de Vida do Objeto com Peewee

1. Introdução

O objetivo deste trabalho é compreender como realizar operações de manipulação de dados utilizando o Peewee ORM em Python. Essas operações incluem a inserção, consulta, atualização e remoção de registros em um banco de dados.

Além das operações CRUD (Create, Read, Update e Delete), o trabalho aborda a persistência dos dados, a validação de informações, o tratamento de exceções e a utilização de sinais (hooks) para executar ações automaticamente durante o ciclo de vida dos objetos.

Para exemplificar os conceitos, será utilizado um sistema de gerenciamento de amigos, no qual cada registro possui um identificador, um nome e um nome de usuário.

2. Objetivos

Os principais objetivos desta atividade são:

- Compreender as operações CRUD utilizando o Peewee.
- Conhecer diferentes formas de inserir registros no banco de dados.
- Realizar consultas e buscas de registros.
- Atualizar e excluir informações persistidas.
- Implementar validações para evitar dados inválidos.
- Tratar exceções durante as operações com o banco de dados.
- Utilizar sinais do Peewee para implementar regras de negócio e auditoria.
- Compreender como ocorre a persistência do estado dos objetos.

3. O que é o Peewee ORM?

O Peewee é uma biblioteca Python que permite interagir com bancos de dados relacionais utilizando classes e objetos, sem precisar escrever manualmente todas as instruções SQL.

ORM significa Object-Relational Mapping, ou mapeamento objeto-relacional. Essa tecnologia associa classes Python a tabelas do banco de dados e atributos dessas classes a colunas das tabelas.

Por exemplo, uma classe chamada Amigo pode representar uma tabela de amigos, enquanto os atributos usuario e nome representam campos dessa tabela.

O Peewee facilita a manipulação dos dados, tornando o código mais organizado e permitindo utilizar métodos específicos para criar, consultar, atualizar e excluir registros.

4. Inserção de dados

A inserção consiste em adicionar novos registros ao banco de dados. O Peewee disponibiliza diferentes métodos para realizar essa operação.

4.1. Método Model.create()

O método create() permite criar e salvar um registro em uma única operação. Exemplo:

```
amigo = Amigo.create(
    usuario="joao123",
    nome="João"
)
```

Nesse exemplo, um novo amigo é cadastrado no banco de dados. O método recebe os valores dos campos e realiza a inserção automaticamente.

Esse recurso é indicado quando se deseja cadastrar um único registro de maneira simples e direta.

4.2. Método Model.save()

O método save() permite salvar um objeto no banco de dados. Quando utilizado em uma instância nova, ele realiza a inserção do registro. Quando utilizado em uma instância que já representa um registro persistido, ele pode salvar as alterações realizadas.

Exemplo:

```
amigo = Amigo(
    usuario="maria123",
    nome="Maria"
)

amigo.save()
```

Nesse caso, primeiro é criado um objeto em memória e, depois, ele é salvo no banco de dados.

A principal diferença em relação ao create() é que o objeto pode ser preparado e ter seus atributos modificados antes da chamada de save().

4.3. Método insert_many()

O método insert_many() permite inserir vários registros em lote, evitando a necessidade de executar uma chamada individual de create() para cada registro.

Exemplo:

```
amigos = [
    {"usuario": "ana123", "nome": "Ana"},
    {"usuario": "pedro123", "nome": "Pedro"},
    {"usuario": "lucas123", "nome": "Lucas"}
]
```

```
Amigo.insert_many(amigos).execute()
```

A lista contém os dados dos registros que serão inseridos. O método insert_many() prepara a operação, enquanto execute() a executa no banco.

A inserção em lote é útil quando se deseja cadastrar uma quantidade maior de registros, como durante a importação de dados.

Entretanto, essa operação não dispara automaticamente os sinais pre_save e post_save para cada registro. Por isso, as validações necessárias devem ser realizadas separadamente, além de serem mantidas as restrições de integridade do banco de dados.

5. Leitura e busca de registros

A leitura permite recuperar informações que já estão armazenadas no banco de dados.

5.1. Método select().where()

O método select() prepara uma consulta, enquanto where() define uma condição para filtrar os registros.

Exemplo:

```
def listar_amigos(usuario):
    return Amigo.select().where(
        Amigo.usuario == usuario
    )
```

Nesse exemplo, a consulta retorna os registros cujo campo usuario corresponde ao valor recebido pela função.

O resultado pode conter zero, um ou vários registros. A consulta é executada quando os resultados são percorridos ou avaliados.

5.2. Método get()

O método get() é utilizado para recuperar um único registro que corresponda a uma condição.

Exemplo:

```
try:
    amigo = Amigo.get(
        Amigo.usuario == "joao123"
    )

    print(amigo.nome)

except Amigo.DoesNotExist:
    print("Amigo não encontrado.")
```

Se o registro não existir, o Peewee gera a exceção DoesNotExist, que pode ser tratada com um bloco try/except.

Esse método é indicado quando se espera encontrar um registro específico.

5.3. Método get_or_none()

O método get_or_none() funciona de maneira semelhante ao get(), mas retorna None quando não encontra o registro.

Exemplo:

```
amigo = Amigo.get_or_none(
    Amigo.usuario == "maria123"
)

if amigo is None:
    print("Amigo não encontrado.")
else:
    print(amigo.nome)
```

Esse método é útil quando a ausência de um registro é uma situação esperada e não é necessário gerar uma exceção para tratá-la.

5.4. Método get_or_create()

O método get_or_create() procura um registro com base nos critérios informados e tenta criá-lo caso ele não exista.

Exemplo:

```
amigo, criado = Amigo.get_or_create(
    usuario="ana123",
    defaults={"nome": "Ana"}
)

if criado:
    print("Amigo cadastrado.")
else:
    print("O amigo já estava cadastrado.")
```

O método retorna dois valores: o objeto encontrado ou criado e um indicador booleano que informa se houve criação.

Para evitar duplicações, é importante utilizar restrições de unicidade no banco de dados, principalmente quando existem operações concorrentes.

6. Atualização de dados

A atualização consiste em modificar informações de registros que já existem no banco de dados.

6.1. Atualização com save()

Para atualizar um registro com save(), é necessário recuperar o objeto, modificar seus atributos e salvar as alterações.

Exemplo:

```
amigo = Amigo.get(
    Amigo.usuario == "joao123"
)

amigo.nome = "João Silva"

amigo.save()
```

Primeiro, o registro é recuperado. Em seguida, seu nome é alterado na memória. Finalmente, o método save() persiste a mudança no banco de dados.

Se a consulta não encontrar o registro, o método get() gera DoesNotExist, que deve ser tratado quando necessário.

6.2. Atualização com update().execute()

O método update().execute() permite atualizar diretamente os registros que correspondem a uma condição, sem precisar carregar cada objeto individualmente.

Exemplo:

```
quantidade = (
    Amigo.update(nome="João Silva")
    .where(Amigo.usuario == "joao123")
    .execute()
)
```

Nesse exemplo:

- update() define o campo e o novo valor.
- where() determina quais registros serão modificados.
- execute() executa a atualização.
- A variável quantidade recebe o número de linhas afetadas, conforme o comportamento do banco de dados.

É importante utilizar uma condição where() quando a intenção for atualizar apenas registros específicos. Sem essa condição, todos os registros da tabela poderão ser modificados.

Diferentemente do salvamento de uma instância, update().execute() não dispara automaticamente os sinais pre_save e post_save.

7. Remoção de dados

A remoção consiste em excluir um registro do banco de dados.

7.1. Método delete_instance()

O método delete_instance() exclui o registro representado por uma instância do modelo.

Exemplo:

```
try:
    amigo = Amigo.get(
        Amigo.usuario == "joao123"
    )

    amigo.delete_instance()

    print("Amigo excluído com sucesso.")

except Amigo.DoesNotExist:
    print("Amigo não encontrado.")
```

Primeiro, o registro é localizado. Depois, delete_instance() executa sua exclusão.

Caso o registro não exista, a busca com get() gera DoesNotExist, permitindo que o erro seja tratado sem interromper inesperadamente o programa.

Esse método também aciona o sinal pre_delete, permitindo registrar informações antes da exclusão.

8. Validações e integridade dos dados

A validação consiste em verificar se os dados recebidos atendem às regras definidas pelo sistema.

Em um sistema de gerenciamento de amigos, algumas regras possíveis são:

- O nome não pode ser vazio.
- O nome de usuário não pode ser vazio.
- O nome de usuário deve ser único.
- Os dados precisam respeitar os tipos e as restrições definidos no modelo.
- As validações ajudam a evitar inconsistências e melhoram a confiabilidade do sistema.

É importante diferenciar a validação realizada em Python das restrições do banco de dados. Por exemplo, verificar se um nome de usuário já existe antes de cadastrá-lo não substitui a restrição unique=True, pois duas operações concorrentes ainda poderiam tentar cadastrar o mesmo valor.

9. Sinais ou hooks do Peewee

Os sinais são mecanismos que permitem executar funções automaticamente durante determinadas operações do ciclo de vida de um objeto.

No Peewee, os sinais pre_save, post_save e pre_delete podem ser utilizados para validar dados e registrar operações de auditoria.

Para utilizá-los, o modelo deve herdar de Model importado de playhouse.signals.

9.1. Sinal pre_save

O sinal pre_save é executado antes de um objeto ser salvo.

Ele pode ser utilizado para validar dados e impedir operações que não atendam às regras do sistema.

Exemplo:

```
from playhouse.signals import pre_save

@pre_save(sender=Amigo)
def validar_amigo(model, instance, created):
    if not instance.usuario.strip():
        raise ValueError(
            "O usuário não pode ser vazio."
        )

    if not instance.nome.strip():
        raise ValueError(
            "O nome não pode ser vazio."
        )
```

Nesse exemplo, o sistema verifica se o nome e o usuário estão preenchidos. Caso algum deles esteja vazio, uma exceção é lançada e o salvamento da instância é interrompido.

O parâmetro instance representa o objeto que será salvo, enquanto created indica se a operação está sendo tratada como criação ou atualização.

9.2. Sinal post_save

O sinal post_save é executado depois que o Peewee conclui a operação de salvamento do objeto.

Ele pode ser utilizado para registrar a criação ou a atualização de um registro.

Exemplo:

```
import logging
from playhouse.signals import post_save

logging.basicConfig(level=logging.INFO)
```

```
@post_save(sender=Amigo)
def registrar_salvamento(model, instance, created):
    acao = "criado" if created else "atualizado"

    logging.info(
        "Amigo %s: id=%s, usuario=%s",
        acao,
        instance.id,
        instance.usuario
    )
```

Quando um amigo é cadastrado ou atualizado, uma mensagem é registrada no log.

O parâmetro created permite distinguir as duas situações. Assim, o sistema consegue manter um histórico básico das operações realizadas.

9.3. Sinal pre_delete

O sinal pre_delete é executado antes da exclusão de uma instância.

Ele pode ser utilizado para registrar qual registro está prestes a ser removido.

Exemplo:

```
from playhouse.signals import pre_delete

@pre_delete(sender=Amigo)
def registrar_exclusao(model, instance):
    logging.info(
        "Amigo prestes a ser excluído: id=%s, usuario=%s",
        instance.id,
        instance.usuario
    )
```

Quando delete_instance() é chamado, a função registra informações sobre o objeto antes de sua exclusão.

9.4. Importância dos sinais

Os sinais permitem separar algumas regras de negócio das operações principais do CRUD. Isso facilita a organização do código e permite executar validações ou registrar eventos automaticamente.

Entretanto, eles não substituem as restrições do banco de dados e não são acionados automaticamente para cada registro nas operações em lote, como insert_many() e update().execute().

Por isso, as regras de negócio e os registros de auditoria precisam ser planejados também para essas operações.

10. Tratamento de exceções

O tratamento de exceções permite lidar com erros previsíveis sem interromper inesperadamente a execução do programa.

10.1. Exceção DoesNotExist

Essa exceção ocorre quando uma consulta que exige um registro, como get(), não encontra correspondência.

Exemplo:

```
try:
    amigo = Amigo.get(
        Amigo.usuario == "usuario_inexistente"
    )
```

except Amigo.DoesNotExist: print("O amigo solicitado não existe.")

O tratamento permite informar que o registro não foi encontrado e continuar a execução do programa.

10.2. Exceção IntegrityError

A exceção IntegrityError pode ocorrer quando uma operação viola uma restrição de integridade do banco de dados, como a tentativa de inserir um nome de usuário duplicado em um campo definido como único.

Exemplo:

from peewee import IntegrityError

```
try:
    Amigo.create(
        usuario="joao123",
        nome="Outro João"
    )

except IntegrityError:
    print(
        "Não foi possível cadastrar: "
        "o usuário pode já estar registrado."
    )
```

Esse exemplo pressupõe que joao123 já exista no banco. A exceção pode indicar outras violações de integridade, portanto, em sistemas reais, é importante registrar os detalhes técnicos do erro para diagnóstico.

10.3. Exceção ValueError

No exemplo deste trabalho, ValueError é utilizado pelo sinal pre_save para indicar que os dados não atendem às regras de validação.

Exemplo:

```
try:
    Amigo.create(
        usuario=" ",
        nome="João"
    )

except ValueError as erro:
    print(f"Dados inválidos: {erro}")
```

O tratamento informa o problema de maneira controlada.

11. Persistência do estado dos objetos

A persistência é a capacidade de manter os dados armazenados no banco mesmo depois que o programa é encerrado.

Quando um objeto é criado em Python, seus atributos ficam inicialmente na memória. Para que seus dados sejam armazenados permanentemente, é necessário realizar a operação de gravação apropriada.

Por exemplo:

```
amigo = Amigo(
    usuario="carlos123",
    nome="Carlos"
)

amigo.save()
```

Após o salvamento, o registro fica armazenado no banco de dados.

Se o programa for encerrado e executado novamente, o registro poderá ser recuperado por meio de uma consulta, desde que o mesmo banco de dados seja utilizado.

A persistência é importante porque garante que as informações não dependam apenas da memória do programa.

12. Exemplo de fluxo completo

Um fluxo de manipulação de dados pode ser organizado nas seguintes etapas:

1. Inicializar a conexão com o banco de dados.
2. Criar as tabelas necessárias.
3. .Cadastrar registros utilizando create() ou save().
4. Inserir vários registros com insert_many(), quando necessário.
5. Consultar os dados com select().where(), get(), get_or_none() ou get_or_create().

6. Validar os dados por meio de regras de negócio.
7. Atualizar registros com save() ou update().execute().
8. Excluir registros utilizando delete_instance().
9. Tratar exceções como DoesNotExist, IntegrityError e ValueError.
10. Registrar eventos relevantes por meio dos sinais e do módulo logging.
11. Confirmar que os dados continuam persistidos após reiniciar o programa.
12. Durante a execução, devem ser testadas situações válidas e inválidas, como cadastrar um usuário duplicado, buscar um registro inexistente, atualizar um registro existente e excluir um registro.
13. Também é necessário verificar que as operações em lote não acionam automaticamente todos os sinais de instância e que as validações pertinentes são realizadas separadamente.

13. Testes recomendados

Para comprovar o funcionamento do sistema, devem ser realizados os seguintes testes:

- Cadastrar um amigo com dados válidos.
- Tentar cadastrar um amigo com nome vazio.
- Tentar cadastrar dois amigos com o mesmo nome de usuário.
- Consultar um amigo existente utilizando get().
- Consultar um amigo inexistente e tratar DoesNotExist.
- Utilizar get_or_none() para verificar a ausência de um registro.
- Utilizar get_or_create() para evitar a criação desnecessária de registros já existentes.
- Atualizar um amigo usando save().
- Atualizar registros utilizando update().execute().
- Excluir um amigo usando delete_instance().
- Verificar os registros de auditoria produzidos pelos sinais.
- Encerrar e executar novamente o programa para verificar a persistência dos dados.
- Esses testes ajudam a demonstrar que as operações CRUD, as validações e o tratamento de erros funcionam de maneira integrada.

14. Conclusão

A manipulação de dados com o Peewee permite realizar operações de criação, consulta, atualização e exclusão de maneira organizada, utilizando objetos Python para representar os registros do banco de dados.

Os métodos create(), save() e insert_many() oferecem diferentes formas de inserção. Os métodos get(), get_or_none() e get_or_create() facilitam as consultas. Já save(), update().execute() e delete_instance() permitem atualizar e remover registros.

O tratamento de exceções contribui para a robustez do sistema, permitindo lidar com situações como registros inexistentes e violações de integridade. Os sinais pre_save, post_save e pre_delete permitem implementar validações e registrar eventos de auditoria durante o ciclo de vida das instâncias.

Por fim, a persistência garante que os dados permaneçam armazenados mesmo após o encerramento do programa. A integração desses recursos resulta em um sistema mais organizado, confiável e alinhado às boas práticas de manipulação de dados com Python e Peewee.